

1. Overview

The game is a turn-based Civilization-like strategy game where structures are built and units are deployed with playing cards from a deck. The initial aim of the game was to have modifiable decks, so that the players would make strategic deck-building choices throughout the game, but deck modifications were left unimplemented due to time constraints. In the game, the player must manage their resources through the use of different buildings providing them, and use units to attack and defend from other players. Playing cards cost resources. In one turn, the player plays down cards on the map and moves and attacks with their placed down units. The goal of the game is to beat other players by destroying their capital building and thus defeating their empire.

The player can interact with the game using their mouse through a visual interface. The game includes a simple main menu from which a game can be created or loaded. A created game can have 2-4 players, different map sizes and different starting decks. During a game, the player can click on cards, buildings and units to manipulate them. The map can be panned with the arrow keys. By pressing the ESC key, the game is saved and the player is returned to the main menu. A game state can be saved to a file and loaded from it to save the player's progress between sessions.

The world is constructed from single color hexagonal tiles while units and buildings are represented by simple shapes. Units are circles and buildings are rectangles, and a health bar is visualized next to them. Multiplayer gameplay is implemented locally with 2-4 players. There is no single player option. The player can only see tiles that are nearby to their already placed buildings or units. Other tiles are shown as only grey and cannot be accessed before they are approached with units or buildings are constructed close to them.

2. Software Structure

The back-end of the game is implemented using a central game class which owns the Map and the Players of the game. The game is responsible for interacting with the UI and advancing the core gameplay loop. The Map owns the individual Tiles. Tiles store their Terrain and Resources. Players own their Deck, which contains a Hand, and both of these contain Cards.

Cards have an abstract base class with currently two subclasses: BuildingCard and UnitCard. These store a Building and a Unit respectively and place the stored Building or Unit on a Tile on the Map when played. Buildings and Units also have a base abstract class with subclasses. Currently units only have a single subclass: Soldier, and buildings have four: CapitalBuilding, FarmBuilding, LumberMillBuilding and MineBuilding. Individual Buildings, Units and Cards from one subclass can still be tweaked with, for example, different hit points, damage, resource multipliers and costs.

Buildings and Units on the Tiles of the Map are shared between the Player owning the Building or Unit and the Tile containing it. The original plan of the game included effects

altering the behaviour of Buildings and Units on Tiles but Effects are currently not implemented.

The user interface of the game is implemented with SFML using a central UI class which handles updating and rendering the UI elements. The main classes responsible for this are MainMenuRenderer, InfoLayerRenderer and MapRenderer, which are owned by the UserInterface class. There are also classes for ui elements such as TextInput, FileSelection and ClickableText. The elements are updated inside the event handling loop when handling clicks etc. and outside of it to handle resizing and hovering effects etc. and then drawn to the window. When the game is not initialized yet, the main menu is shown where you can either start a new game or load a game from file. After starting a new game or loading a game from a save file, the game object is initialized and the map and info layer are rendered. When returning to the main menu, the game object is deinitialized and the main menu is then rendered again.

3. Instructions for building and using the software

The program requires SFML 2.5.1 to work and must be run using WSL.

Download the required SFML graphics library on WSL using

sudo apt install libsfml-dev

More info in <https://www.sfml-dev.org/tutorials/2.5/start-linux.php>

Note: the SFML version available from the Ubuntu repos is 2.5.1 on Intel machines and 2.6.1 on AMD machines, but both work.

To build and run the program using CMake run the following commands

cmake -S . -B build

cmake --build build

./build/cpp_game

Or alternatively by launching it from the VS Code run and debug menu using, for example, the option:

(gdb) Launch (strategy-game-3)

4. Testing

The basic game logic modules (Game, Player, Map, Building, Unit, Card) were tested by running simple unit tests written in /tests. Separate VS code configurations for running and debugging the tests was implemented to ease testing and finding errors. The tests were mainly used to catch errors in basic functions of the game logic classes and to monitor the Valgrind output of the game logic side.

In more practical terms, a separate test program (test_main.cpp) which logs the successful and failed tests and their outcomes was implemented. The test program was designed with scalability in mind so that it would be easy to add more test functions to the program as the project progresses. This made it easy for testers to simply write a new testing function using custom assertion functions (AssertWithMessage) and then add the test function to the test program when the function was ready.

```

test_suite tests[] = {
    {"Testing Game Initialization and turns",
tests::TestGameInitializationAndTurns},
    {"Testing Game Save and Load", tests::TestGameSaveAndLoad},
    {"Testing Player class", tests::TestPlayer},
    {"Testing Map class", tests::TestMap},
    {"Testing Buildings", tests::TestBuildings},
    {"Testing Units", tests::TestUnits},
    {"Testing Cards", tests::TestCards},
};

```

Test run outcomes using the above functions:

Tests run:

###: Testing Game Initialization and turns

###: ---> PASSED

###: Testing Game Save and Load

###: ---> PASSED

###: Testing Player class

###: ---> PASSED

###: Testing Map class

###: ---> PASSED

###: Testing Buildings

###: ---> PASSED

###: Testing Units

###: ---> PASSED

###: Testing Cards

###: ---> PASSED

Tests passed: 7/7

The UI was tested visually by each developer. This means that when implementing a new feature, the developer would test it straight in the UI and check if the implementation works as expected. By inspecting the changes in UI, all major errors were captured early on and fixed before leading to other errors. Separate UI unit tests were considered by the team but deemed unnecessary due to successful visual debugging.

5. Work log

Division of work:

- Oliver
 - Main focus: Both UI & logic
 - Game and Player class logic
 - Game saving and loading logic (excluding Map and Cards)
 - Unit testing and a separate test suite
 - Info layer implementation and linking to game
 - Writing and updating meeting notes

- Aleksi
 - Main focus: UI
 - Main menu implementation
 - User interface class implementation
 - Helper classes in UI
 - SFML research and setup for the project
- Topi
 - Main focus: UI
 - Map rendering implementation
 - Building and unit rendering and logic
 - Tile selection logic
 - Initial main menu
 - SFML research and setup
- Konsta
 - Main focus: Logic
 - Card, Hand and Deck classes
 - Creating initial header files and documentation for classes and functions
 - Software structure
- Atte
 - Main focus: Logic
 - Map, Tile, Building and Unit implementation
 - Map saving and loading

In sprint 1 the focus was on establishing the basics of the game logic and main menu. The work was mainly divided into UI work (Aleksi & Topi) and game logic work (Oliver & Atte & Konsta).

	Task	Completed	Developer(s)	Time used
Sprint 1	Main menu with SFML	x	Aleksi, Topi	8 h each
	Establish basic map logic	x	Atte	
	Initial Game class implementation	x	Oliver	4 h
	Initial Player class implementation	x	Oliver	4-6 h
	Implement basics of Terrain class	x	Atte?	
	Header files	x	Konsta	8 h

In the beginning of sprint 2 the team struggled with some CMake issues related to SFML linking and project building which were fortunately quickly solved. After the slow start, the focus moved on implementing a working map in UI to ease debugging. The game logic side also progressed with the implementation of initial Building, Unit and Card classes. Basics of game saving & loading were also implemented.

	Task	Completed	Developer(s)	Time used
--	------	-----------	--------------	-----------

Sprint 2	Initial game state saving & loading	x	Oliver	6 h
	Look into unit testing	x	Oliver	8 h
	Implement unit class	x	Atte	
	Implement building class	x	Atte	
	Implement terrain graphics with SFML	x	Topi	8 h
	Create UI classes and refactor UI code	x	Aleksi	8 h
	Implement Card class		Konsta	0 h

The goal of Sprint 3 was to make the game interactive which required implementing the most important remaining game logic classes and connecting them to the existing game logic and further to UI. The goal of interactivity also required contributions to the UI side. These include an Info layer enabling game turn changes and being able to select tiles from the rendered map to display their info.

	Task	Completed	Developer(s)	Time used
Sprint 3	Implement Card class	x	Konsta	5 h
	Refine game map graphics and add interactivity on SFML	x	Topi	6 h
	Implement general UI classes and refactor UI code	x	Aleksi	15 h
	Implement UI info layer	x	Oliver	15 h
	Merge Building and Unit class to master	x	Atte	
	Implement Hand and Deck classes	x	Konsta	7 h
	Buildings UI	x	Topi	3 h

In Sprint 4 the goal was to finally make the game playable by adding more interactivity through map panning, card playing, unit movement & attacking and selection highlighting. Making the game playable also includes finalizing the main menu and game saving & loading functionality in the main menu.

	Task	Completed	Developer(s)	Time used
Sprint 4	Implement Unit and Building	x	Konsta	2 h

	player and tile setting			
	Add Cards to info layer	x	Oliver	6 h
	Test Cards, Buildings and Units	x	Oliver	4 h
	Map distance method	x	Atte?	
	Map panning	x	Topi	4 h
	Map saving & loading	x	Atte	
	Add Card costs	x	Konsta	2 h
	Better map generation	x	Atte	
	Map generation methods	x	Atte	
	Player sight UI	x	Topi	3 h
	Make Building and Unit classes virtual	x	Atte	
	Implement Unit movement & attacking in UI	x	Oliver	2 h
	Unit saving & loading	x	Oliver	3 h
	Add health bars to Buildings & Units	x	Oliver	1 h
	Add variety to Buildings and Units	x	Konsta	5 h
	Add individual Cards	x	Konsta	3 h
	Implement Card, Hand and Deck saving and loading	x	Konsta	5 h
	Units UI	x	Topi	2 h
	Add resizing support to main menu	x	Aleksi	5 h
	Add file selection to load game screen	x	Aleksi	6 h
	Improve main menu	x	Aleksi	6 h
	UserInterface class improvements	x	Aleksi	3 h
	Add cool title animation	x	Aleksi	3 h